Here we have defined a function called `write()`, which works in much the same way as the previous example, but now we have added a parameter called `$in` to the function declaration. PHP will now expect that when `write` is called, either literal data, a variable, or any expression that evaluates to the expected type will be included as an argument. This is also known as "passing in" data. We call `write()` twice, once with a string literal and once with a variable holding string data.

We also have another function: `glue()`. Rather than printing within its block, it takes two inputs – `$start` and `$end` – and makes a single string from them. It then uses the `return` keyword to send this data back to the calling code. When `glue()` is called, the sequence of events happens like this:

- PHP stops executing the calling code
- Incoming data is assigned to the function parameter variables
- PHP executes the function block until it encounters the `return` statement
- Execution of the calling code resumes using the result of `glue()` in the remainder of the expression
- The `print` statement executes with the returned string data

It helps if you read `print glue('hello', 'again');` from right to left: first we encounter the `glue()` function call, then we encounter `print`. By the time `print` executes, `glue()` is done and we have our string, as if we'd actually written `print "hello again\n";`.

Scalar Type Coercion

In Chapter 2, "Data, Types, and Variables," we saw how PHP chooses data types for you when scalar variables are declared and assigned a value. In a similar spirit of convenience and helpfulness, PHP will attempt to manage type casting automatically when scalar types don't strictly match their operations. Type coercion also occurs in function calls.

To give a quick example, the built-in function `trim()` operates on strings by removing whitespace and returning the result. From the PHP documentation for the `trim` function, the function signature is `trim(string $string, string $characters = " \n\r\t\v\0"): string`. The main thing to note is `trim(string $string ...): string`,